

Checklist Geral de Implementação — SIGES (Baseado no CDU V5)

Este documento consolida o **checklist completo de tudo o que foi implementado até o momento** no projeto SIGES, confrontando cada função e componente técnico desenvolvido com as exigências e regras de negócio especificadas no **CDU V5 (Documento de Casos de Uso Mais Recente)**.

Para cada item implementado, é detalhado: 1. **Identificação e Status:** Se está concluído e homologado ou planejado. 2. **Função / Componente Técnico:** Nome da função no código, rota e arquivo de localização (`backend/main.py` , `backend/db.py` ou `frontend/app.js`). 3. **O que foi feito na prática:** Como a funcionalidade foi construída e se comporta no sistema. 4. **O que ela deveria fazer (Regra do CDU V5):** O requisito formal de engenharia e negócio segundo a especificação.

Índice dos Módulos

- 1. Fundações de Infraestrutura e Performance (Bloco 1)
- 2. Motor Colaborativo: Timeline e Notificações (Bloco 2)
- 3. Módulo de Pendências, Hierarquia e SLA (Bloco 3)
- 4. Fluxos de Tela 01 e Lotes Inteligentes (Bloco 4)
- 5. Perfis de Tela, Layouts Dinâmicos e RBAC (CDU V4 / V5)
- 6. Auditoria Global e Ingestão de Dados (CDU V3 / V5)
- 7. Roadmap Comparativo: O que falta implementar (Bloco 5)

1. Fundações de Infraestrutura e Performance (Bloco 1)

Objetivo no CDU V5: Suportar bases massivas do GPM com centenas de milhares de linhas sem travamento de tela ou latência de carregamento (Item 5.1 do CDU V5).

✓ CONCLUÍDO 1.1 Paginação Server-Side e Lazy Loading

- Função / Componente Técnico:**
 - Backend: `listar_servicos()` via rota `GET /api/servicos` e consulta parametrizada em `buscar_servicos_db()`.
 - Frontend: `Store.carregarServicosAPI()` e renderizador visual `renderPaginador()`.
- O que foi feito:**
 - O backend aceita os parâmetros `skip` e `limit` (padrão 10 itens por página), executa uma consulta SQL com `LIMIT %s OFFSET %s` e um `COUNT(*)` paralelo com os mesmos filtros ativos.
 - Retorna o formato estruturado `{ "data": [ ... ], "total": X }`.
 - O frontend renderiza controles de paginação numérica com botões anterior/próximo e atalhos rápidos.
- O que deveria fazer segundo o CDU V5:**
 - O grid dinâmico principal deve carregar e processar bases massivas com centenas de milhares de linhas sem congelamento do navegador, renderizando sob demanda apenas a fatia solicitada (Item 5.1).

✓ CONCLUÍDO 1.2 Persistência de Sessão no Frontend

- Função / Componente Técnico:**
 - Frontend: `Store.constructor()` e métodos de sincronização com chave `localStorage.getItem('siges_filtros_v1')`.
- O que foi feito:**
 - Todos os estados voláteis da interface — contrato ativo, filtros de busca, cargo/responsável selecionado, página atual e ordenação da tabela — são gravados em tempo real no `localStorage`.
 - Ao carregar a página ou pressionar **F5**, o estado é reconstituído sem que o usuário perca seu contexto de trabalho.
- O que deveria fazer segundo o CDU V5:**
 - Reter parâmetros de navegação (filtros, ordenações, largura de colunas e paginação corrente) ao alternar telas ou recarregar a janela, garantindo ergonomia operacional contínua (Item 5.1).

✓ CONCLUÍDO 1.3 Integridade dos Indicadores do Dashboard (KPIs Globais)

- Função / Componente Técnico:**
 - Backend: Rota `GET /api/dashboard/kpis` mapeada em `backend/main.py`.
 - Frontend: `Store.carregarKPIsAPI()` e renderização dos cards e gráficos em `renderGerencial()`.
- O que foi feito:**
 - A rota de KPIs executa agregações completas no MySQL (`COUNT(*)`) agrupado por `status_id`, desacoplada da paginação de 10 linhas da tabela.

- **O que deveria fazer segundo o CDU V5:**

- O card "Serviços na Esteira" e os gráficos analíticos devem refletir a totalidade real dos serviços da filial/contrato, garantindo visão gerencial precisa mesmo com lazy loading na listagem.

✓ CONCLUÍDO **1.4 Desacoplamento de Chaves: num_servico vs tdc**

- **Função / Componente Técnico:**

- Banco de Dados: Colunas `num_servico` e `tdc` tratadas como identificadores independentes em `backend/db.py`.
- Frontend: Colunas separadas nas tabelas e no Drawer de detalhes `renderDrawer()`.

- **O que foi feito:**

- Tratamento individual das duas chaves, suportando relações em que múltiplos serviços compartilham o mesmo TDC ou vice-versa.

- **O que deveria fazer segundo o CDU V5:**

- `num_servico` (Chave Primária GPM) e `tdc` (Chave Distribuidora) possuem independência estrutural e suportam relações de cardinalidade 1:N, 1:1, N:1 e N:M, servindo o TDC para cross-reference com sistemas externos (Item 1.1).

2. Motor Colaborativo: Timeline e Notificações (Bloco 2)

Objetivo no CDU V5: Centralizar a colaboração humana ativa entre Medição, Operação, Validação e Faturamento em histórico imutável (Item 1.0 e 5.2 do CDU V5).

✓ CONCLUÍDO **2.1 Modelagem da Timeline e Persistência Imutável**

- **Função / Componente Técnico:**

- Backend: `inserir_comentario_db()` e `buscar_comentarios_db()`.
- Rotas: `GET /api/servicos/<id>/comentarios` e `POST /api/servicos/<id>/comentarios` em `backend/main.py`.
- Banco de Dados: Tabela `comentarios_servico` contendo `servico_id`, `usuario_id`, `usuario_nome`, `texto` e `created_at`.

- **O que foi feito:**

- O backend grava comentários com chave estrangeira para o serviço e o usuário logado, com carimbo automático de data/hora (timestamp). Não há rota de edição ou exclusão de comentários.

- **O que deveria fazer segundo o CDU V5:**

- Registrar histórico cronológico e centralizado imutável de interações humanas associando autor, perfil e data/hora, nunca permitindo deleção lógica ou física (Item 1.0 e 5.2).

✓ CONCLUÍDO **2.2 Interface da Timeline no Painel Lateral (Drawer)**

- **Função / Componente Técnico:**

- Frontend: Container `#timeline-container` dentro de `renderDrawer()` e chamadas assíncronas no evento de abertura da SOB.

- **O que foi feito:**

- Ao clicar em qualquer serviço na tabela, o Drawer abre e dispara imediatamente o fetch para a rota de comentários daquela SOB específica, exibindo indicador de loading e mensagens em formato de balões com autor e horário.

- **O que deveria fazer segundo o CDU V5:**

- Exibir o histórico de discussões e anotações técnicas em formato de chat/timeline dentro do detalhe do serviço (Item 1.0 e 1.2).

✓ CONCLUÍDO **2.3 Formatador Visual de Menções (@usuario)**

- **Função / Componente Técnico:**

- Frontend: Expressão regular no parser de texto do comentário em `frontend/app.js`.

- **O que foi feito:**

- Textos com o padrão `@nome_usuario` são capturados e renderizados com tag visual estilizada (badge com cor destacada).

- **O que deveria fazer segundo o CDU V5:**

- Suportar marcações de usuários (`@usuario`) para identificação de destinatários e direcionamento de notificações (Item 1.0).

✓ CONCLUÍDO **2.4 Segregação Estrita de Conversas por SOB**

- **Função / Componente Técnico:**

- Frontend e Backend: Chave de filtro `servico_id` obrigatória em `buscar_comentarios_db()`.

- **O que foi feito:**

- Limpeza obrigatória do container de timeline ao alternar entre serviços, garantindo que o histórico de uma SOB nunca apareça em outra.

- **O que deveria fazer segundo o CDU V5:**

- Garantir a integridade do dossiê do serviço, evitando que apontamentos técnicos vazem entre ordens de serviço distintas.

3. Módulo de Pendências, Hierarquia e SLA (Bloco 3)

Objetivo no CDU V5: Regras rígidas para o trabalho de campo, controle de prazos e fluxo hierárquico na Tela 02 (Item 1.1, 2 - Tela 02 e Seção 3 do CDU V5).

✓ CONCLUÍDO 3.1 Filtragem Hierárquica Automática por Perfil (RN-05)

- **Função / Componente Técnico:**
 - Frontend: Leitura de cargo em `AuthService.getPerfilAtivo()` e aplicação de filtro default no `Store.carregarServicosAPI()`.
 - Backend: Cláusulas dinâmicas `WHERE s.supervisor = %s` ou `WHERE s.coordenador = %s` em `buscar_servicos_db()`.
- **O que foi feito:**
 - Ao fazer login como Supervisor, o sistema pré-seleciona seu nome no filtro e restringe a grid às equipes subordinadas a ele. Ao logar como Coordenador, filtra por seus supervisores.
 - Disponibilizada a opção "Todos" (Limpar Filtro) para consulta ampla.
- **O que deveria fazer segundo o CDU V5:**
 - Ao acessar a Tela de Pendências, o grid é pré-filtrado de forma automática com base no login e perfil da árvore de subordinação (Coordenador vê subordinados; Supervisor vê equipes diretas), com prerrogativa de desfiltração para consultas gerais (Item 2 - Tela 02).

✓ CONCLUÍDO 3.2 Tabelas Auxiliares Configuráveis de Pendências

- **Função / Componente Técnico:**
 - Backend: `obter_parametros_pendencias_db()` e rota `GET /api/parametros/pendencias`.
 - Frontend: `Store.carregarParametrosPendenciasAPI()` e renderização das opções em `renderMedicao()`.
- **O que foi feito:**
 - Removidos todos os textos fixos (hardcodes) do frontend. As opções de desvios são carregadas diretamente do banco de dados MySQL (`itens_correcao_cosampa` e `itens_correcao_distribuidora`).
 - Categorias organizadas:
 - **Cosampa:** 1. Fotos (Baixa qualidade, Sem evidência); 2. Materiais (Aplicados/Retirados incorretamente); 3. Documentação (Sem Croqui, Croqui incorreto).
 - **Distribuidora:** Vozes não cadastradas, Serviço não despachado, Ordem já faturada.
- **O que deveria fazer segundo o CDU V5:**
 - O backend e os componentes visuais devem ser alimentados diretamente pelas tabelas de parametrização lógica administráveis (Seção 3 do CDU V5).

✓ CONCLUÍDO 3.3 Tramitação em Lote e Individual para Pendências (Status 02 e 04)

- **Função / Componente Técnico:**
 - Frontend: `Store.tramitarLoteAPI()` vinculado aos botões `#btn-enviar-pend-cosampa` e `#btn-enviar-pend-distribuidora`.
 - Backend: Rota `POST /api/servicos/lote/tramitar` em `backend/main.py`.
- **O que foi feito:**
 - O operador pode selecionar um ou dezenas de serviços na Medição, marcar os itens de correção correspondentes e enviá-los em lote para o Status `02` (Pendências Cosampa) ou `04` (Pendências Distribuidora). O serviço é retirado da Medição e passa a constar na Tela 02.
- **O que deveria fazer segundo o CDU V5:**
 - "Gerar Pendência Cosampa" envia para `02. PENDENCIAS OPERACIONAIS COSAMPA` e "Gerar Pendência Distribuidora" para `04. PENDENCIAS DISTRIBUIDORA`. A Pendência Cosampa tem prioridade em caso de apontamento duplo (Item 2 - Tela 01 e Matriz 4.1).

✓ CONCLUÍDO 3.4 Bloqueio com Trava de Reprogramação e Alerta de SLA

- **Função / Componente Técnico:**
 - Frontend: Eventos e validações no Drawer de saneamento em `renderPendencias()` e `renderDrawer()`.
 - Backend: Persistência em `atualizar_dados_servico_db()`.
- **O que foi feito:**
 - Inclusão do checkbox "Necessita Reprogramar Visitada em Campo". Caso marcado, o botão de salvar trava e emite alerta visual exigindo o preenchimento da `data_programacao`.
 - Cálculo dinâmico de dias decorridos exibido no badge: 🕒 **Atraso SLA: X dias**.
 - Botão de retorno tramita o serviço sanado de volta para `01. AGUARDANDO CONFERENCIA`.
- **O que deveria fazer segundo o CDU V5:**
 - Checkbox "Necessita Reprogramar" exige obrigatoriamente a indicação de uma nova Data de Programação para retorno da equipe. Monitoramento de SLA comparando data de execução com data de entrada em pendência (Item 2 - Tela 02).

✓ CONCLUÍDO 3.5 Ordenação Temporal nas Pendências

- **Função / Componente Técnico:**
 - Backend: Cláusula `ORDER BY COALESCE(s.dta_exec_srv, s.created_at) DESC` em `buscar_servicos_db()`.
- **O que foi feito:**
 - Os serviços na tela de pendências são ordenados do mais recente para o mais antigo com fallback seguro para colunas existentes no schema (`dta_exec_srv` / `created_at`).
- **O que deveria fazer segundo o CDU V5:**
 - Facilitar a triagem e o tratamento das pendências mais urgentes e recentes pela supervisão técnica.

4. Fluxos de Tela 01 e Lotes Inteligentes (Bloco 4)

Objetivo no CDU V5: Blindagem de operações em lote com falha parcial, validação mandatória de origem, bypass comercial acelerado e correlação de serviços irmãos na Medição (Item 1.1, 2 - Tela 01 e 5.3 do CDU V5).

✓ CONCLUÍDO 4.1 Validação Obrigatória do Sistema de Origem (Trava da Tela 01)

- **Função / Componente Técnico:**
 - Backend: Trava em `tramar_servico_db()` e `origem_sistema` retornada por `buscar_servicos_db()`.
 - Frontend: Seletor de Origem com botão de salvamento no Drawer `renderDrawer()`, seletor em lote `#select-bulk-origem` em `renderMedicao()` e trava impeditiva no botão `#drawer-btn-tramar`.
- **O que foi feito:**
 - Ao tentar tramitar qualquer serviço da Medição (status 01) para status operacional posterior sem que o campo `origem_sistema` esteja confirmado, o sistema bloqueia a ação e emite um alerta explícito exigindo a seleção da origem (ex: Eorder, Synergia, SacBt, PDA).
 - Na tabela, serviços sem origem exibem o badge em destaque vermelho **NÃO VALIDADO**. O Drawer e a barra lateral permitem salvar o Sistema de Origem individualmente ou em lote.
- **O que deveria fazer segundo o CDU V5:**
 - Antes de qualquer envio para o próximo status, o sistema exige a validação/confirmação do campo SISTEMA DE ORIGEM DO SERVIÇO, exibindo o valor default contido na tabela principal e permitindo correções em lote ou individuais (Item 1.1 e 2 - Tela 01).

✓ CONCLUÍDO 4.2 Bypass do Fluxo Comercial (Salto de Status Direto 01 → 08)

- **Função / Componente Técnico:**
 - Backend: Cláusula de desvio condicional em `tramar_servico_db()` acionada por `contrato`, `tipo_servico` ou `tipo_obra`.
 - Injeção na Timeline: `inserir_comentario_db()` registrando aviso oficial de bypass na SOB.
 - Frontend: Botão `#btn-aprovar-medicao` em `renderMedicao()`.
- **O que foi feito:**
 - Ao aprovar a medição de serviços cujo contrato ou tipo seja Comercial, o backend intercepta o avanço e redireciona o serviço diretamente para o **Status 08 (Validado Aguardando Faturamento)**, sem passar pelo Status 03 (Fechamento).
 - A Timeline do serviço registra um comentário automático auditável informando o salto de status. Serviços do fluxo Emergencial avançam normalmente para o Status 03.
- **O que deveria fazer segundo o CDU V5:**
 - Serviços do fluxo Emergencial aprovados avançam para **03. AGUARDANDO ENVIO PARA VALIDACAO**. Serviços do fluxo Comercial aprovados avançam diretamente para **08. VALIDADO AGUARDANDO AUTORIZACAO DE FATURAMENTO** (Item 2 - Tela 01 e Matriz de Transição 4.1).

✓ CONCLUÍDO 4.3 Lotes Inteligentes com Mecânica de Falha Parcial

- **Função / Componente Técnico:**
 - Backend: Rotas `/api/servicos/lote/tramar` e `/api/servicos/lote/enviar-validacao` retornando código HTTP 200 com resumo `{ "sucessos": X, "sucessos_ids": [...], "falhas": Y, "erros": [...] }`.
 - Frontend: `Store.tramarLoteAPI()` e modal `#btn-modal-cdu02`.
- **O que foi feito:**
 - Ao tramitar múltiplos serviços em lote, cada item é validado individualmente. Se 1 serviço de um lote de 100 apresentar erro (ex: falta de Sistema de Origem ou pendência não saneada), os 99 regulares são tramitados com sucesso no banco.
 - O sistema não devolve erro 500 nem cancela o lote inteiro. Um sumário informativo exibe a quantidade de sucessos e lista detalhadamente os serviços com erro e seus motivos. Os serviços com falha continuam selecionados na tela para intervenção rápida do operador.
- **O que deveria fazer segundo o CDU V5:**
 - Caso haja qualquer registro inválido por pendências técnicas ou ausência de informações, o lote total não deve falhar. O sistema deve reter especificamente o registro inconsistente, exibindo sumário com os erros identificados e persistindo no fluxo de alteração todos os demais registros regulares (Item 5.3).

✓ CONCLUÍDO 4.4 Correlação de Serviços Irmãos e Hyperlinks GPM

- **Função / Componente Técnico:**
 - Frontend: Algoritmo de agrupamento e detecção em `renderMedicao()` e renderizador de linhas `renderDynamicTableRow()`.
- **O que foi feito:**
 - O sistema analisa em tempo real os serviços da tabela e identifica correlações que compartilham a mesma Incidência, mesmo PEP/Obra ou mesmo Id do Cliente.
 - Serviços correlacionados recebem borda lateral azul destacada (`border-left: 3.5px solid #0284c7`), fundo suave e o badge  SOB Irmã com tooltip detalhando o motivo do vínculo.
 - As colunas de ID / Número da Ordem agora são **hyperlinks azuis clicáveis** (`target="_blank"`) com ícone de link externo, direcionando diretamente para a visualização da ordem no sistema legado GPM sem interromper a navegação da esteira.
- **O que deveria fazer segundo o CDU V5:**
 - Exibir correlação de Serviços Irmãos agrupando ordens por Incidência/Obra (Emergencial) ou Incidência/Cliente (Comercial), além de atalhos e links diretos clicáveis para abrir o módulo de ordens no GPM (Item 1.1 e 2 - Tela 01).

5. Perfis de Tela, Layouts Dinâmicos e RBAC (CDU V4 / V5)

Objetivo no CDU V5: Visualização customizável por usuário e segregação estrita de acesso baseada em perfis (Item 1 e 5.2 do CDU V5).

✓ CONCLUÍDO 5.1 Gerenciamento de Perfis de Tela (Layouts de Colunas)

- **Função / Componente Técnico:**
 - Backend: Rotas `/api/perfis_tela` (GET, POST, PUT), `/api/perfis_tela/todos` e `/api/usuarios/<id>/perfis_tela` em `backend/main.py`.
 - Frontend: `renderToolbarPerfis()`, `renderDynamicTableHeaders()` e `renderDynamicTableRow()`.
- **O que foi feito:**
 - Usuários podem selecionar perfis de visualização (ex: "Padrão", "Operacional", "Fechamento"), criar novos layouts de colunas, reordenar e salvar preferências individuais ou globais.
- **O que deveria fazer segundo o CDU V5:**
 - Permitir que os usuários criem, salvem, modifiquem e disponibilizem layouts personalizados de colunas para quaisquer outros usuários da plataforma, mantendo a estrutura padrão de identificação rápida (Item 1 e 1.1).

✓ CONCLUÍDO 5.2 Controle de Acesso e Permissões Baseadas em Papéis (RBAC)

- **Função / Componente Técnico:**
 - Backend: Rota `POST /api/auth/login`, rotas de usuários `/api/usuarios` e perfis `/api/perfis` em `backend/main.py`.
 - Frontend: Classe `AuthService` e modal administrativo `renderGestaoAcessos()`.
- **O que foi feito:**
 - Autenticação com perfis definidos (Master, Analista de Fechamento, Coordenador, Supervisor). Telas laterais e botões de ação são habilitados ou bloqueados conforme a matriz de permissões.
 - Inclusão do recurso de criação e **edição de usuários existentes** diretamente pela interface administrativa.
- **O que deveria fazer segundo o CDU V5:**
 - Restrição estrita de acesso baseada em perfis (RBAC). A navegação de supervisores deve ser voltada às telas operacionais de pendência, limitando transições financeiras e de faturamento a perfis gerenciais e analistas de fechamento (Item 5.2).

6. Auditoria Global e Ingestão de Dados (CDU V3 / V5)

Objetivo no CDU V5: Rastreabilidade imutável de todas as modificações e carga dinâmica de dados (Item 1.0 e 1.2 do CDU V5).

✓ CONCLUÍDO 6.1 Motor Global de Auditoria Imutável (Audit Log)

- **Função / Componente Técnico:**
 - Backend: `registrar_log_auditoria()` e `buscar_logs_auditoria()`.
 - Rota: `GET /api/servicos/<id>/auditoria` em `backend/main.py`.
 - Frontend: Seção de Logs de Auditoria no Drawer de detalhes.
- **O que foi feito:**
 - Qualquer alteração de status ou atualização de campo dispara a gravação no MySQL com: data/hora, usuário, e-mail, campo alterado, valor anterior e novo valor.
- **O que deveria fazer segundo o CDU V5:**
 - Qualquer alteração realizada no sistema, em qualquer campo (mudanças de status, valores de input, justificativas), deve obrigatoriamente gerar um registro imutável de Log de Auditoria capturando data, hora, usuário, ID do serviço, campo, valor anterior e novo valor (Item 1.0 e 5.2).

✓ CONCLUÍDO 6.2 Importador Dinâmico Inteligente (Auto-Sync com De-Para)

- **Função / Componente Técnico:**
 - Backend: `processar_importacao_dinamica()` e rotas `/api/servicos/upload-temp`, `/api/servicos/pre-visualizar`, `/api/servicos/importar-dinamico`.
 - Frontend: `renderWizardImportacao()`.
- **O que foi feito:**
 - Assistente de importação de planilhas Excel/CSV com detecção de abas e cabeçalhos, mapeamento interativo de colunas De-Para, ignorando células vazias para não sobrescrever dados prévios e sincronizando com o banco.
- **O que deveria fazer segundo o CDU V5:**
 - Permitir a ingestão massiva e padronizada de dados operacionais e de execução vindos de fontes externas ou do GPM (Item 1.2).

7. Roadmap Comparativo: O que falta implementar (Bloco 5)

Abaixo está o quadro de funcionalidades previstas no **CDU V5** que correspondem ao último bloco do plano de implementação e que **ainda não foram implementadas**:

Status	Funcionalidade (CDU V5)	O que a função deverá fazer	Bloco Previsto
 PENDENTE	Telas Dedicadas de Faturamento e Conciliação (Telas 04 e 05)	Criar interfaces dedicadas para gerir status do 08 ao 13, com inputs de Data de Validação e Mês de Emissão (Item 2 - Telas 04 e 05).	Bloco 5
 PENDENTE	Evento Transacional de Conciliação e Snapshot "Relatório ANTES"	Processar planilhas de pagamento da Distribuidora com geração de snapshot pré-importação, direcionando serviços 100% batidos para Finalizado e divergências para 11 (Item 2 - Tela 05).	Bloco 5
 PENDENTE	Comparador Dinâmico Bilateral (Status 11)	Interface comparando Valor Realizado vs. Valor Pago por atividade na conciliação, exigindo justificativa obrigatória na Timeline para tramitação (Item 2 - Tela 01, Status 11).	Bloco 5
 PENDENTE	Gestão de Disputas e Pagamento a Menor (Status 12 e 13)	Controle de SLA com Mês de Reapresentação da Medição e acompanhamento de custódia de disputa contratual com o cliente (Item 2 - Tela 01, Status 12 e 13).	Bloco 5

Resumo Executivo de Progresso

- **Bloco 1 (Fundações e Performance):** 100% Concluído e Homologado.
- **Bloco 2 (Motor Colaborativo / Timeline):** 100% Concluído e Homologado.
- **Bloco 3 (Módulo de Pendências / SLA / Hierarquia):** 100% Concluído e Homologado.
- **Bloco 4 (Fluxos Tela 01 e Lotes Inteligentes):** 100% Concluído e Homologado.
- **Transversais (Perfis de Tela, RBAC, Auditoria, Importador):** 100% Concluído e Homologado.
- **Bloco 5 (Módulo Financeiro e Conciliação):** Planejado (Aguardando liberação de desenvolvimento).